Department of Computer Science and Engineering

(AI & ML)

B. Tech. CSE - 5th Semester

Aug – Dec 2025

UE23CS351A

DATABASE MANAGEMENT SYSTEMS (DBMS)

PROJECT REPORT on

COLDCHAIN GUARDIAN

Database-Driven Monitoring and Analysis for Temperature-Sensitive Shipments

Submitted by:

Akash Ramesh PES1UG24AM212 Backend APIs and database integration	Pratham S Reddy PES1UG24AM200 Frontend UI and user experience, integration, and testing

Table of Contents

Abstract

1. Introduction

2. User Requirements Specification

3. E-R Model

4. Relational Model

5. SQL DDL Statements

6. SQL DML Statements

7. NoSQL and Vector Database Commands

8. Results

9. Next.js Project Tree Structure

10. Sample Screenshots of Application Code and Demo

11. Evaluation Rubrics

Abstract

ColdChain Guardian is a web application for monitoring refrigerated shipments and reviewing historical cold-chain data. It combines a relational MySQL database for operational logistics with MongoDB for sensor readings, login sessions, and a separate published strawberry-transport research dataset.

The application supports role-based sign-in, shipment management, temperature and humidity checks, alert review, and operational reports. A separate research report analyzes 14,398 historical windows from six source shipments. It summarizes probe temperatures, missing measurements, risk labels, and user-defined temperature excursions. Historical research records remain separate from live operational shipments. The exposure index is a deterministic descriptive calculation, not a forecast.

For a selected temperature range, the analysis reports the percentage of valid probe measurements outside the bounds, together with excursion and missing-reading rates for each of nine probe positions. This exposes spatial variation and data completeness in the historical shipment without claiming to predict future spoilage.

1. Introduction

1.1 Problem Statement

Temperature-sensitive products require controlled conditions during transportation. Temperature variations can affect product quality, making shipment monitoring and timely alerts important. ColdChain Guardian addresses this through a database-driven system that tracks shipments, monitors sensor readings, identifies limit violations, and maintains shipment history.

1.2 Short Description

ColdChain Guardian allows authorized users to manage shipments, monitor temperature and humidity readings, and review alerts. MySQL stores logistics data, while MongoDB stores sensor readings and historical research data. A separate research module analyzes strawberry transportation data to identify temperature excursions and missing measurements across nine probe positions.

2. User Requirements Specification

2.1 Technology Stack and Justification

The application combines a responsive web interface, server APIs, secure authentication, and two databases selected for their different data needs.

Layer	Technology	Use in the project
Web application	Next.js 16.4.3, React 19.2.8, TypeScript	App Router pages, server APIs, and interactive UI
Styling	Tailwind CSS 4	Responsive layouts and component styling
Relational database	MySQL 8+	Users, products, routes, shipments, items, history, and alerts
Document database	MongoDB	Sensor readings, sessions, and research windows
Authentication	bcryptjs and opaque session tokens	Password hashes in MySQL; hashed session tokens in MongoDB
Runtime	Node.js 20.19+	Next.js application and database scripts

The stack supports a responsive interface and server-side processing while keeping structured logistics records in MySQL and flexible sensor or research documents in MongoDB.

2.2 Justification for Using MongoDB (NoSQL)

Sensor readings and historical research records contain flexible document structures and optional fields. MongoDB stores each observation with its shipment, sensor, timestamp, temperature, and humidity values, while also accommodating the source-shaped research windows and import metadata.

The project uses four MongoDB collections:

- sensor_readings stores operational sensor observations.
- research_strawberry_windows stores published historical strawberry-transport data.
- research_dataset_imports stores the pinned source revision, hash, dimensions, and import metadata.
- user_sessions stores SHA-256 hashes of opaque session tokens and their expiry dates.

MySQL manages structured operational data and relationships; MongoDB handles sensor observations, sessions, and historical research documents. A vector database or vector-search index is not used. The application uses relational queries and MongoDB filters and aggregation pipelines.

3. E-R Model

3.1 Entities and Relationships

The conceptual ER diagram models the logistics and alerting domain. Its principal entities are suppliers, warehouses, drivers, vehicles, users, routes, shipments, products, shipment items, non-operational shipments, alerts, shipment status history, and shipment assignments. Shipment items associate products with shipments; assignments associate users with shipment work. Routes connect source and destination warehouses, and shipment status changes are retained in history.

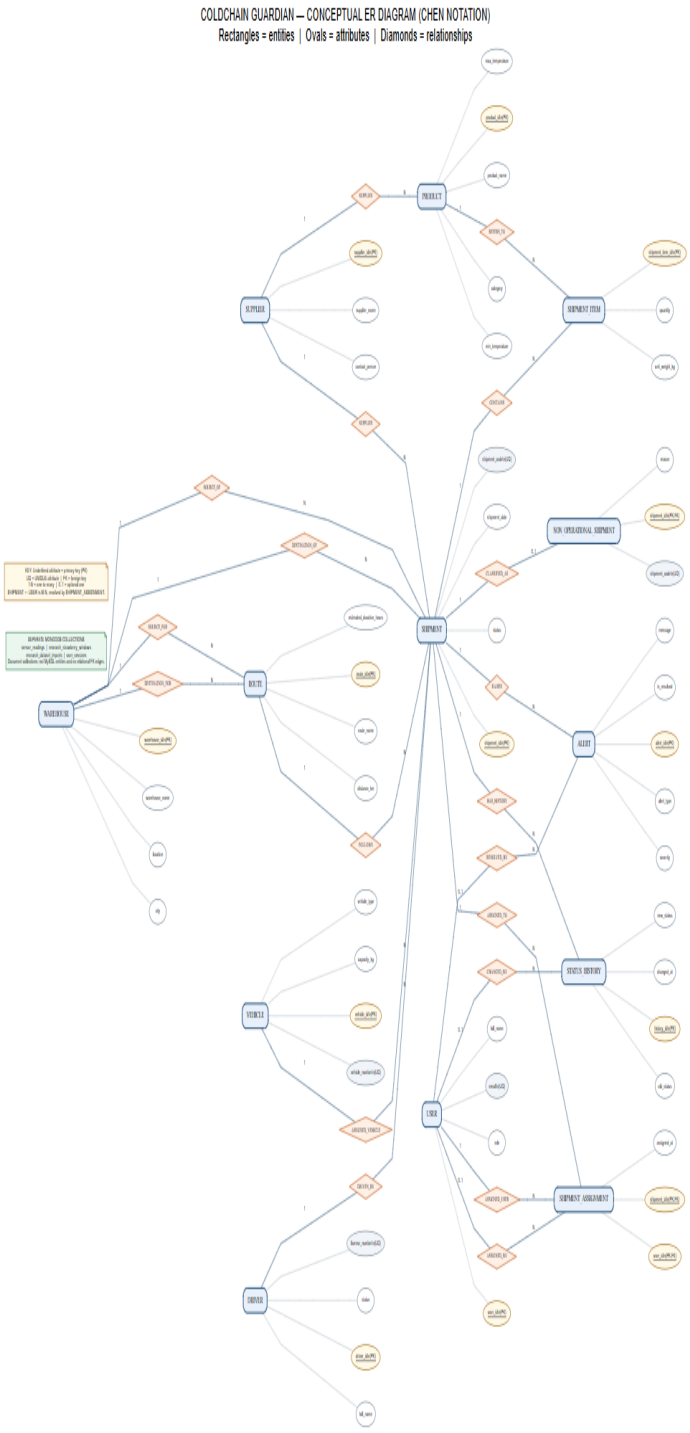


Figure 1. ColdChain Guardian conceptual ER diagram (Chen notation).

4. Relational Model

4.1 Relations and Integrity Rules

The relational schema maps the operational entities to 13 MySQL base tables. Primary keys identify rows; foreign keys preserve links among shipments, suppliers, warehouses, routes, products, staff, vehicles, drivers, alerts, and history. Constraints enforce positive shipment-item quantities and weights, unique shipment-product pairs, valid status values, and other declared checks. MongoDB collections remain separate from this relational schema.

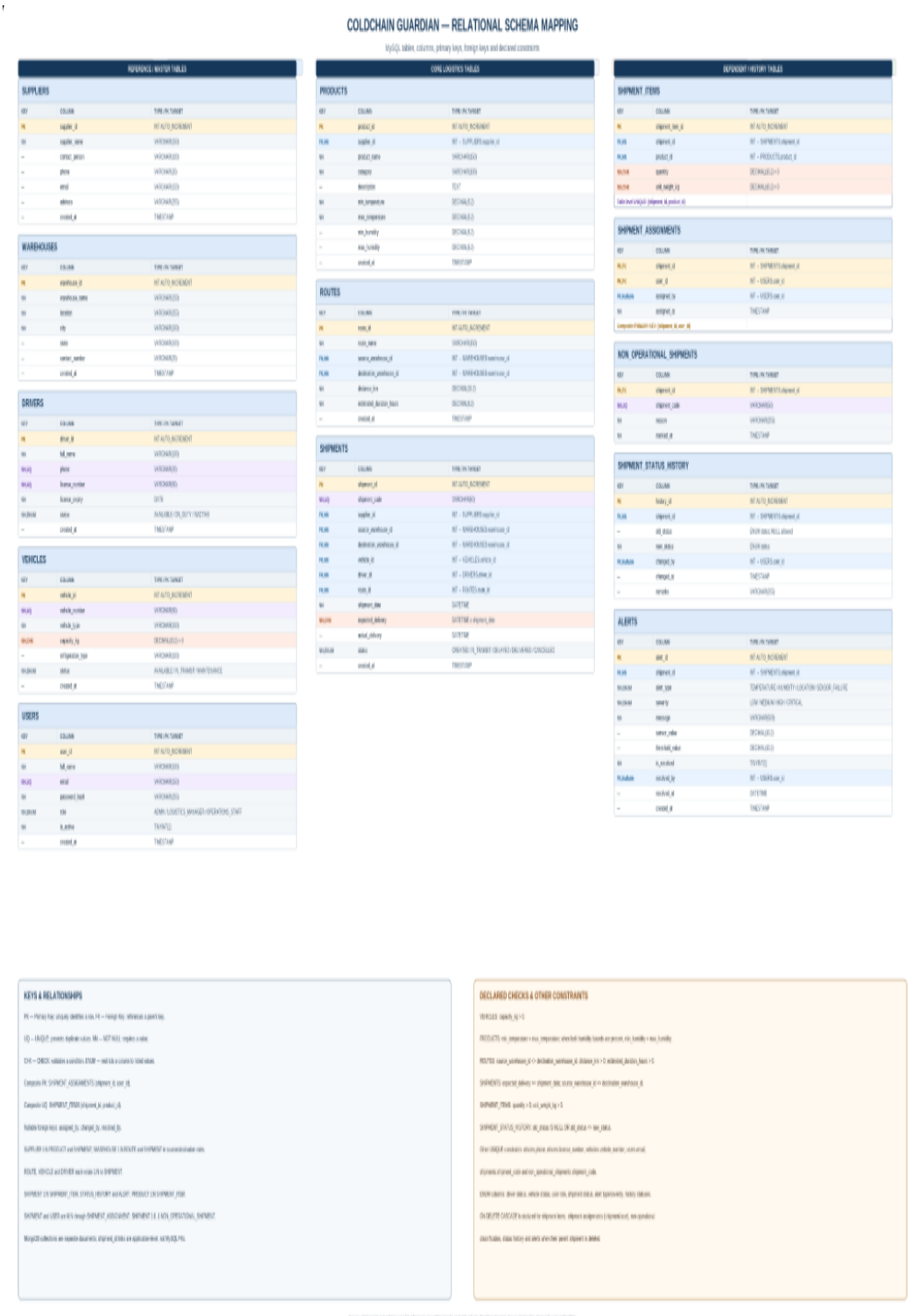


Figure 2. Relational schema mapping with MySQL tables, keys, and constraints.

5. SQL DDL Statements

5.1 Create Table with Constraints

```
CREATE TABLE shipment_items (
 shipment_item_id INT AUTO_INCREMENT PRIMARY KEY,
 shipment_id INT NOT NULL,
 product_id INT NOT NULL,
 quantity DECIMAL(10,2) NOT NULL CHECK (quantity > 0),
 unit_weight_kg DECIMAL(10,2) NOT NULL CHECK (unit_weight_kg > 0),
 UNIQUE (shipment_id, product_id),
 FOREIGN KEY (shipment_id) REFERENCES shipments(shipment_id),
 FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

5.2 Create View

```
CREATE OR REPLACE VIEW shipment_overview AS
SELECT s.shipment_id, s.shipment_code, sup.supplier_name,
 sw.warehouse_name AS source_warehouse,
 dw.warehouse_name AS destination_warehouse, s.status
FROM shipments AS s
JOIN suppliers AS sup ON sup.supplier_id = s.supplier_id
JOIN warehouses AS sw ON sw.warehouse_id = s.source_warehouse_id
JOIN warehouses AS dw ON dw.warehouse_id = s.destination_warehouse_id;
```

5.3 Trigger, Stored Procedure, and Schema

- The schema contains 13 MySQL base tables with primary keys, foreign keys, and constraints.
- The trg_shipment_status_history trigger records a row when a shipment status changes.
- The sp_create_alert stored procedure provides a reusable alert insertion operation using supplied shipment and sensor details.

The current temperature-check API also creates alerts transactionally and deduplicates unresolved conditions.

6. SQL DML Statements

6.1 Insert

```
INSERT INTO suppliers (supplier_name)
VALUES ('Example supplier');
```

6.2 Grouped Operational Report Query

```
SELECT s.status, COUNT(*) AS shipment_count
FROM shipments AS s
WHERE NOT EXISTS (
 SELECT 1 FROM non_operational_shipments AS n
 WHERE n.shipment_id = s.shipment_id
)
GROUP BY s.status
ORDER BY s.status;
```

The exclusion subquery keeps classified classroom or demo shipments out of operational totals while preserving their source rows.

6.3 Join Query

```
SELECT s.shipment_code, r.route_name,
 sw.warehouse_name AS source,
 dw.warehouse_name AS destination
FROM shipments AS s
JOIN routes AS r ON r.route_id = s.route_id
JOIN warehouses AS sw ON sw.warehouse_id = s.source_warehouse_id
JOIN warehouses AS dw ON dw.warehouse_id = s.destination_warehouse_id
WHERE s.shipment_id = ?;
```

6.4 Update and Delete

```
UPDATE shipments SET status = ?
WHERE shipment_id = ? AND status = ?;

DELETE FROM suppliers WHERE supplier_id = ?;
```

The API validates input and uses parameterized SQL. A delete succeeds only when the applicable foreign-key rules allow it.

6.5 Recursive CTE for Route Reachability

```
WITH RECURSIVE route_walk AS (
 SELECT w.warehouse_id AS origin_id, w.warehouse_id AS current_id,
 0 AS hops,
 CAST(CONCAT('/', w.warehouse_id, '/') AS CHAR(2000)) AS visited,
 CAST(w.warehouse_name AS CHAR(2000)) AS path
 FROM warehouses w
 WHERE w.warehouse_id = (SELECT MIN(warehouse_id) FROM warehouses)
 UNION ALL
 SELECT rw.origin_id, r.destination_warehouse_id, rw.hops + 1,
 CONCAT(rw.visited, r.destination_warehouse_id, '/'),
 CONCAT(rw.path, ' -> ', d.warehouse_name)
 FROM route_walk rw
 JOIN routes r ON r.source_warehouse_id = rw.current_id
 JOIN warehouses d ON d.warehouse_id = r.destination_warehouse_id
 WHERE rw.hops < 4
 AND LOCATE(CONCAT('/', r.destination_warehouse_id, '/'), rw.visited) = 0
)
SELECT origin_id, current_id, hops, path
FROM route_walk WHERE hops > 0;
```

6.6 Transaction Pattern

Shipment creation writes the shipment and related records, reserves the vehicle and driver, inserts shipment items, and records initial status history in one transaction. The API rolls the operation back on error.

```
START TRANSACTION;
-- Insert shipment, items, staff assignments, and initial history.
-- Reserve the vehicle and driver.
COMMIT;
-- On error, the API rolls back the transaction.
```

7. NoSQL and Vector Database Commands

MongoDB stores timestamped operational sensor events and source-shaped historical research windows. It also stores hashed login sessions and dataset import metadata. Vector search is not used in this project.

7.1 Collections and Example Document

Collection	Purpose
sensor_readings	Operational temperature, humidity, location, and device readings
research_strawberry_windows	Published historical strawberry measurements and labels
research_dataset_imports	Pinned source revision, hash, dimensions, and import metadata
user_sessions	SHA-256 hashes of random session tokens with expiry dates

```
{
 "shipment_id": 123,
 "sensor_id": "container-probe-1",
 "timestamp": ISODate("2026-10-31T10:20:00Z"),
 "temperature": 5.2,
 "humidity": 55
}
```

7.2 Index Setup and Read Commands

```
// Idempotent project index setup
npm run setup:mongodb

// Read operational observations for one shipment
db.sensor_readings.find(
 { shipment_id: 123 },
 { _id: 0, timestamp: 1, temperature: 1, humidity: 1 }
).sort({ timestamp: 1 });
```

7.3 Aggregate Temperature Statistics

```
db.sensor_readings.aggregate([
 { $group: {
 _id: "$shipment_id",
 readings: { $sum: 1 },
 meanTemperature: { $avg: "$temperature" },
 minimum: { $min: "$temperature" },
 maximum: { $max: "$temperature" }
 } },
 { $sort: { _id: 1 } }
]);
```

7.4 Create, Update, and Delete

```
// Create or update a stable sensor observation (upsert).
db.sensor_readings.updateOne(
 { _id: 'operational:stable-reading-key' },
 { $set: { shipment_id: 123, sensor_id: 'probe-1',
 timestamp: ISODate('2026-10-31T10:20:00Z'),
 temperature: 5.2, humidity: 55 } },
 { upsert: true }
);

// Logout or revocation removes the matching hashed session.
db.user_sessions.deleteOne({ token_hash: '<sha256-token-hash>' });
```

Application APIs validate sensor upserts. Session deletion supports logout and revocation; session documents store token hashes rather than raw tokens.

8. Results

8.1 Operational Database and Application

MySQL stores normalized operational records for users, products, routes, shipments, shipment items, status history, and alerts. The project includes a shipment overview view, a status-history trigger, and an alert procedure. The application reports shipment status and monthly counts, unresolved alert severity, route totals, and average delay. Sensor summaries combine MongoDB observations with MySQL product thresholds.

8.2 Historical Strawberry Research Analysis

The separate research report covers 14,398 historical windows from six source shipments and nine probe positions. In the displayed analysis, the user-selected range is 10 °C to 20 °C. Across the returned windows, the mean recorded T_mean is 15.060 °C; the available probe mean is 10.890 °C across 68,706 non-missing measurements. There are 1,195 rows without a risk label. The UI reports a spatial exposure index of 88.9%, 12,526 windows with excursions, and 61,083 out-of-range probe measurements. These are descriptive results for the chosen bounds, not a prediction.

Rank	Probe position	Valid readings	Excursions	Excursion rate	Missing values
1	Rear Bottom	8,610	8,136	94.5%	5,788 (40.2%)
2	Middle Bottom	4,253	4,007	94.2%	10,145 (70.5%)
3	Front Top	7,724	7,220	93.5%	6,674 (46.4%)
4	Rear Top	7,512	6,860	91.3%	6,886 (47.8%)
5	Front Middle	10,216	9,298	91.0%	4,182 (29.0%)
6	Rear Middle	8,111	7,325	90.3%	6,287 (43.7%)
7	Middle Middle	7,081	6,027	85.1%	7,317 (50.8%)
8	Middle Top	8,808	7,333	83.3%	5,590 (38.8%)
9	Front Bottom	6,391	4,877	76.3%	8,007 (55.6%)

Across the source dataset, the future risk label counts shown are 10,675 positive, 2,333 negative, and 1,390 missing. The interface preserves missing labels and reports missing measurements by probe so readers can interpret the analysis alongside data completeness.

9. Next.js Project Tree Structure

The project uses the Next.js App Router. Page components provide the shipment dashboard, alerts, reports, login, and administrator settings. MySQL stores ADMIN, LOGISTICS_MANAGER, and OPERATIONS_STAFF roles. Authenticated APIs enforce role permissions and shipment assignments. Passwords use bcrypt hashes in MySQL, while MongoDB stores hashed opaque session tokens.

```
coldchain-guardian/
|-- app/
| |-- page.tsx # shipment dashboard
| |-- login-form.tsx # sign-in UI
| |-- alerts/page.tsx # alerts and resolution UI
| |-- reports/page.tsx # operational and research reports
| |-- settings/page.tsx # admin employee management
| |-- components/app-shell.tsx # navigation and session context
| `-- api/
| |-- auth/ # login, logout, current user
```

```
| |-- users/ # employee administration
| |-- shipments/ # shipment and sensor APIs
| |-- alerts/ # alert list and resolution
| |-- catalog/ # reference-data APIs
| |-- reports/ # operational report aggregates
| `-- research/strawberry/ # historical analysis API
|-- database/
| |-- mysql-schema.sql
| `-- migrations/
|-- lib/ # auth, MySQL, MongoDB, research logic
|-- scripts/ # setup, import, bootstrap, sample data
|-- docs/ # API, ERD, data model, SQL, verification
|-- tests/ # backend, schema, research checks
|-- public/
|-- package.json
`-- .env.example
```

9.1 Employee Password Change

Administrators can open Change password beside an employee account in Settings. The form requires a new password of at least 12 characters and a matching confirmation. The PATCH /api/users/{id} handler bcrypt-hashes the new password and revokes the employee’s active sessions. The account-list API never returns password hashes.

10. Sample Screenshots of Application Code and Demo

The supplied report includes screenshots of the sign-in page, employee settings, alert monitoring, and the published research analysis. They show the implemented screens and sample/demo data.

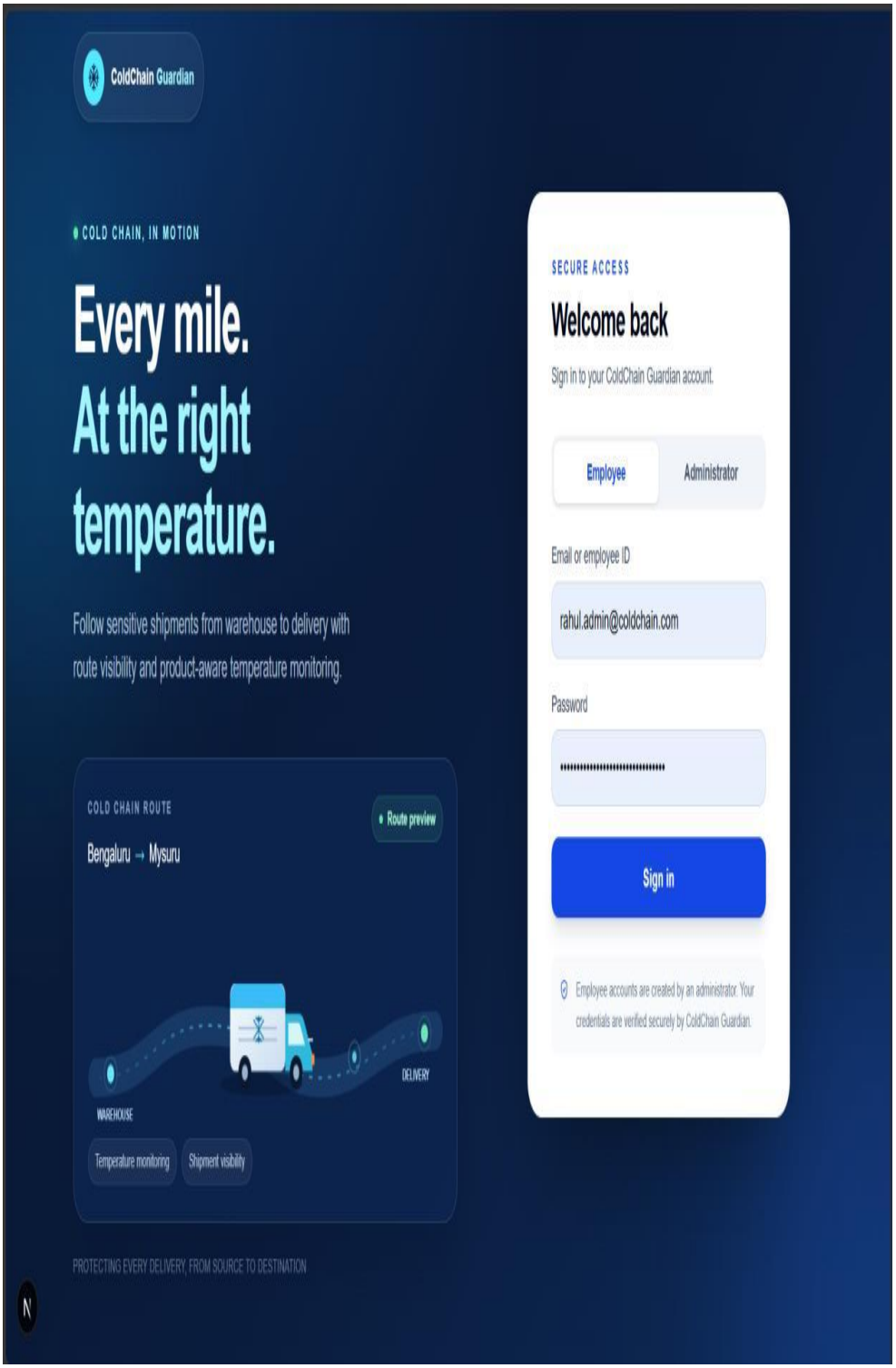


Figure 3. ColdChain Guardian employee and administrator sign-in page.

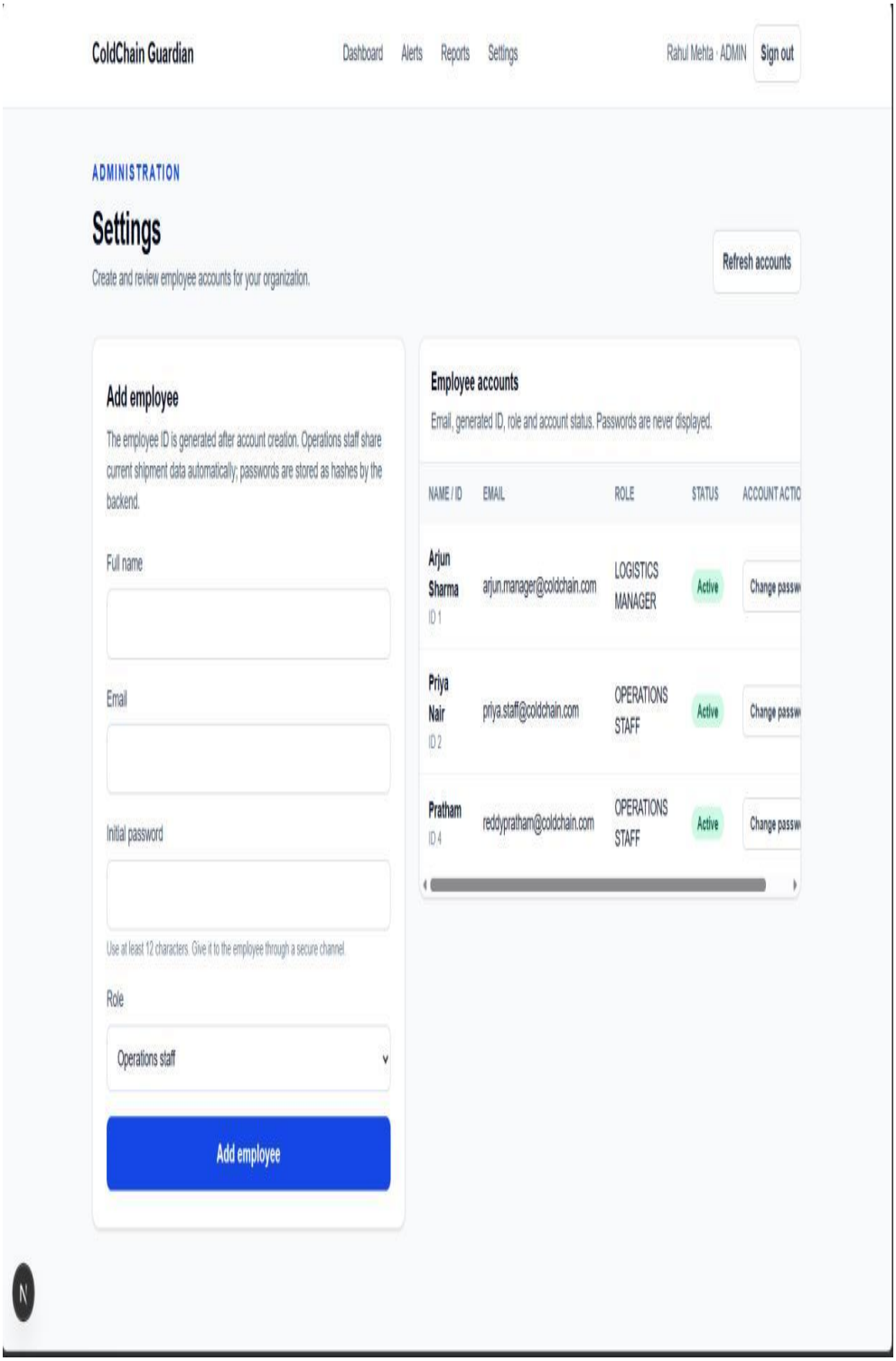


Figure 4. Administrator settings with employee account management.

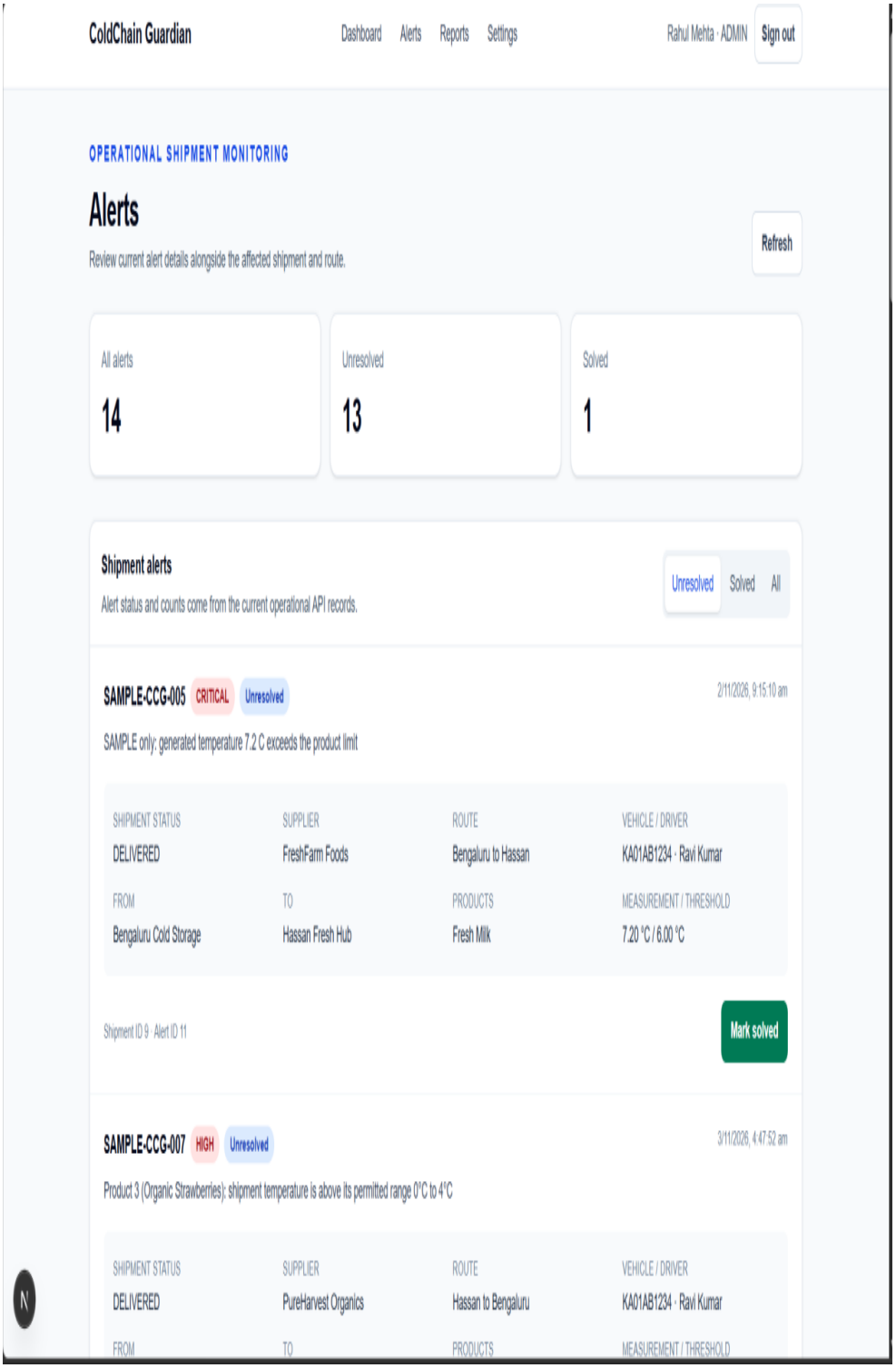


Figure 5. Alert monitoring with unresolved shipment conditions.

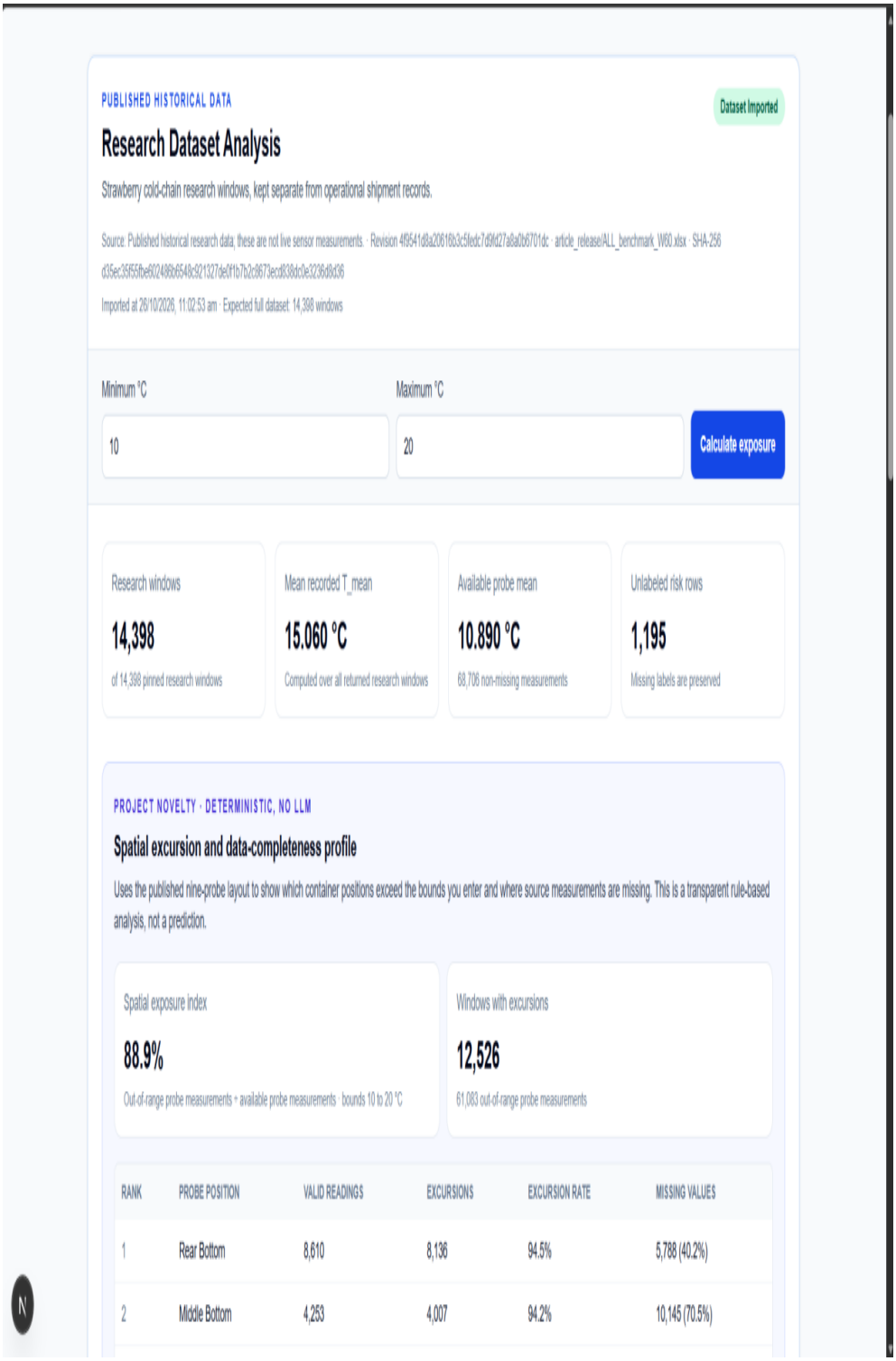


Figure 6. Historical strawberry research analysis and probe excursion summary.



Figure 7. Probe-level excursion rates and research-window distribution.

11. Evaluation Rubrics

The template's evaluation criteria are retained below. Completion-status entries and score distributions are left blank because the supplied report does not provide an evaluated rubric or awarded marks.

#	Criteria	Distribution	Status
1	Database Design	ER diagram, relational schema with keys, relationships, and constraints	Done

#	Criteria	Distribution	Status
2	Database Creation	SQL DDL with constraints, NoSQL database creation, technology justification	Done
3	SQL Operations and Queries	DML operations, nested/correlated/recursive queries, joins	Done
4	Advanced Database Concepts	Procedure, trigger, CTE, NoSQL create/read/update/delete	Done
5	Web Application and Database Integration	Next.js interface, backend/API and database integration	Done
6	Authentication and Authorization	Authentication and role-based access for at least two roles	Done
7	Documentation, Git/GitHub, Demonstration, Testing, and Originality	Documentation, Git/GitHub, originality, and individual contribution	Done